

INFO145 - Diseño y Análisis de Algoritmos

Demostraciones por Inducción en Diseño y Análisis de Algoritmos

Héctor Ferrada

académico

Instituto de Informática - UACH

Introducción

En el análisis y diseño de algoritmos es muy frecuente utilizar demostraciones por inducción. Sin embargo, no siempre se trata exactamente del mismo tipo de inducción. En algunos casos se utiliza inducción matemática clásica sobre una variable numérica, por ejemplo para resolver o acotar ecuaciones de recurrencia. En otros casos se utiliza una forma de inducción asociada al estado de ejecución de un algoritmo iterativo, donde el objetivo es demostrar que cierta propiedad se mantiene verdadera después de cada iteración del algoritmo.

Aunque ambas técnicas comparten la misma idea general —demostrar una propiedad inicialmente y luego probar que se preserva paso a paso—, la forma en que se aplican y la interpretación de la hipótesis inductiva son diferentes. En este documento se estudian ambos escenarios con detalle y mediante ejemplos clásicos de la literatura de algoritmos.

Inducción sobre ecuaciones de recurrencia

Las ecuaciones de recurrencia aparecen de manera natural al analizar algoritmos recursivos. Una recurrencia expresa el tiempo de ejecución de un problema de tamaño n en función del tiempo requerido por instancias más pequeñas del mismo problema.

Considere la siguiente recurrencia, que se estudió en las slides 5_recurrencias.pdf:

$$T(n) = 8T\left(\frac{n}{2}\right) + n^2$$

con:

$$T(1) = 1$$

y suponiendo que n es potencia de 2.

Antes de demostrar formalmente una solución mediante inducción, normalmente se intenta descubrir una posible forma cerrada de la recurrencia. Una técnica muy utilizada consiste en expandir repetidamente la ecuación hasta identificar un patrón general.

Comenzando desde la recurrencia original:

$$T(n) = n^2 + 8T\left(\frac{n}{2}\right)$$

Aplicando nuevamente la recurrencia sobre $T(n/2)$:

$$T(n) = n^2 + 8 \left(\frac{n^2}{4} + 8T \left(\frac{n}{4} \right) \right)$$

Distribuyendo:

$$T(n) = n^2 + 2n^2 + 8^2 T \left(\frac{n}{2^2} \right)$$

Repitiendo el proceso:

$$T(n) = n^2 + 2n^2 + 8^2 \left(\frac{n^2}{2^4} + 8T \left(\frac{n}{2^3} \right) \right)$$

$$T(n) = n^2 + 2n^2 + 2^2 n^2 + 8^3 T \left(\frac{n}{2^3} \right)$$

Continuando sucesivamente, se observa el siguiente patrón:

$$T(n) = n^2 + 2n^2 + 2^2 n^2 + 2^3 n^2 + \dots + 2^{k-1} n^2 + 8^k T \left(\frac{n}{2^k} \right)$$

Ahora elegimos:

$$k = \lg n$$

de modo que:

$$\frac{n}{2^k} = 1$$

Entonces:

$$8^k T \left(\frac{n}{2^k} \right) = 8^{\lg n} T(1)$$

Como:

$$T(1) = 1$$

y además:

$$8^{\lg n} = (2^3)^{\lg n} = 2^{3 \lg n} = 2^{\lg n^3} = n^3$$

obtenemos:

$$8^k T \left(\frac{n}{2^k} \right) = n^3$$

Por lo tanto:

$$T(n) = n^2 + 2n^2 + 2^2 n^2 + \dots + 2^{\lg n - 1} n^2 + n^3$$

Expresando la suma mediante notación sigma:

$$T(n) = \sum_{k=0}^{\lg n - 1} 2^k n^2 + n^3$$

Factorizando n^2 :

$$T(n) = n^2 \sum_{k=0}^{\lg n - 1} 2^k + n^3$$

La suma geométrica:

$$\sum_{k=0}^m 2^k = 2^{m+1} - 1$$

permite obtener:

$$T(n) = n^2(2^{\lg n} - 1) + n^3$$

Como:

$$2^{\lg n} = n$$

resulta:

$$T(n) = n^2(n - 1) + n^3$$

$$T(n) = 2n^3 - n^2$$

Por consiguiente:

$$T(n) = \Theta(n^3)$$

El desarrollo anterior permite *conjeturar* una forma explícita para la recurrencia. Sin embargo, expandir términos no constituye por sí solo una demostración formal. Lo que realmente se ha obtenido es una hipótesis razonable acerca de la solución exacta de la recurrencia. Por este motivo, ahora se utiliza inducción matemática para demostrar rigurosamente que la expresión obtenida es correcta.

Queremos demostrar que:

$$T(n) = 2n^3 - n^2$$

La demostración se realiza por inducción matemática sobre n .

Base inductiva.

Para $n = 1$:

$$T(1) = 1$$

y la fórmula propuesta entrega:

$$2(1)^3 - (1)^2 = 2 - 1 = 1$$

Por lo tanto, la fórmula es correcta para el caso base.

Hipótesis inductiva.

Supongamos que para algún n potencia de 2 se cumple:

$$T(n) = 2n^3 - n^2$$

Debemos demostrar que entonces también se cumple para el siguiente tamaño válido.

Paso inductivo.

Usando la definición de la recurrencia:

$$T(2n) = 8T(n) + (2n)^2$$

Aplicando la hipótesis inductiva:

$$T(2n) = 8(2n^3 - n^2) + 4n^2$$

Distribuyendo:

$$T(2n) = 16n^3 - 8n^2 + 4n^2$$

$$T(2n) = 16n^3 - 4n^2$$

Factorizando en función de $2n$:

$$T(2n) = 2(2n)^3 - (2n)^2$$

que coincide exactamente con la fórmula propuesta.

Por lo tanto, por inducción matemática:

$$T(n) = 2n^3 - n^2$$

para todo n potencia de 2.

Es importante notar que en este tipo de demostraciones la hipótesis inductiva corresponde a asumir correcta la fórmula para tamaños menores del problema. Luego se reemplaza dicha hipótesis dentro de la propia recurrencia para verificar que la expresión también se cumple para tamaños más grandes. Este mecanismo es muy distinto al utilizado en algoritmos iterativos, donde la inducción se aplica sobre el estado interno del algoritmo.

Inducción en algoritmos iterativos

Cuando se demuestra la correctitud de un algoritmo iterativo, normalmente no se utiliza inducción sobre una fórmula numérica, sino sobre las iteraciones del algoritmo. La herramienta principal para este tipo de demostraciones es el concepto de *invariante de ciclo*.

Un invariante de ciclo es una propiedad que permanece verdadera antes y después de cada iteración del algoritmo. La demostración de correctitud consiste entonces en probar tres elementos:

- El invariante es verdadero antes de comenzar el ciclo.
- Si el invariante es verdadero antes de una iteración, sigue siendo verdadero después de ejecutarla.
- Cuando el algoritmo termina, el invariante implica que la solución obtenida es correcta.

Esta estructura corresponde exactamente a una demostración inductiva. La diferencia es que ahora la inducción se realiza sobre las iteraciones del algoritmo y no sobre un número natural arbitrario.

Caso de estudio: Insertion Sort

Considere el algoritmo Insertion Sort:

```
for(int i=1; i<n; i++){
    int key = A[i];
    int j = i-1;

    while(j>=0 && A[j] > key){
        A[j+1] = A[j];
        j--;
    }

    A[j+1] = key;
}
```

La idea del algoritmo consiste en mantener una parte izquierda del arreglo ordenada e ir insertando cada nuevo elemento en su posición correcta.

Para demostrar formalmente su correctitud se utiliza el siguiente invariante:

Antes de comenzar la iteración i , el subarreglo $A[0..i-1]$ está ordenado.

La demostración comienza verificando la inicialización. Antes de la primera iteración se tiene $i = 1$, por lo que el subarreglo considerado es:

$$A[0..0]$$

Un arreglo con un solo elemento siempre está ordenado. Por lo tanto, el invariante es verdadero inicialmente.

Luego se analiza la mantención del invariante. Suponga que antes de cierta iteración el subarreglo:

$$A[0..i-1]$$

está ordenado. El algoritmo toma el elemento $A[i]$ y desplaza hacia la derecha todos los elementos mayores que él. Finalmente inserta dicho elemento en la posición correcta. Como el subarreglo previo ya estaba ordenado y el nuevo elemento se inserta en el lugar adecuado, el resultado es que:

$$A[0..i]$$

queda ordenado al finalizar la iteración. Por lo tanto, el invariante se mantiene.

Finalmente, cuando el ciclo termina se tiene:

$$i = n$$

y el invariante afirma que:

$$A[0..n-1]$$

está ordenado. Ese subarreglo corresponde al arreglo completo, por lo que el algoritmo es correcto.

Obsérvese que la estructura de la demostración es completamente inductiva. Se prueba un caso inicial, luego se demuestra que la propiedad se preserva iteración tras iteración, y finalmente se utiliza dicha propiedad para concluir la correctitud del algoritmo.

Caso de estudio: Algoritmo de Kruskal

El algoritmo de Kruskal permite obtener un árbol recubridor mínimo (*Minimum Spanning Tree*, MST) en un grafo no dirigido y ponderado. La idea general es ordenar las aristas de menor a mayor costo e ir agregándolas siempre que no formen ciclos.

Una versión simplificada del algoritmo de kruskal visto en clases es la siguiente:

```
Ordenar las aristas por peso creciente

T = {}

while (|T| < n-1) do{
    e = arista de menor peso disponible

    if (e no forma ciclo en T) then
        agregar e a T
}
```

La demostración de correctitud se basa nuevamente en inducción sobre las iteraciones del algoritmo. El invariante utilizado es el siguiente:

Después de cada iteración, el conjunto de aristas seleccionado por Kruskal es subconjunto de algún MST del grafo.

Inicialmente:

$$T = \emptyset$$

y el conjunto vacío es subconjunto de cualquier MST. Por lo tanto, el invariante es verdadero al comienzo.

Suponga ahora que el invariante es verdadero antes de cierta iteración; es decir, que la solución parcial T de Kruskal si es subconjunto de un MST válido, llámese $\mathcal{T}$, y ahora Kruskal selecciona una arista:

$$e = (u, v)$$

que no forma ciclo en T . Queremos demostrar que:

$$T \cup \{e\}$$

también es subconjunto de algún MST (ya sea $\mathcal{T}$ u otro).

Bien, si la arista e pertenece a $\mathcal{T}$, entonces no hay nada que demostrar. El caso interesante ocurre cuando e no pertenece a dicho $\mathcal{T}$ pero T si.

Como $\mathcal{T}$ conecta todos los vértices, dentro de él ya existe un camino entre u y v . Si quisiéramos agregar e se formaría un ciclo en $\mathcal{T}$. En ese ciclo debe existir alguna arista e' que conecta dos componentes que Kruskal todavía no había unido en T .

Dado que Kruskal selecciona siempre la arista disponible de menor peso que no forma ciclo, necesariamente se cumple:

$$\omega(e) \leq \omega(e')$$

Si los pesos son iguales, reemplazar e' por e produce otro MST con el mismo costo total, demostrando que Kruskal es correcto.

Pero si:

$$\omega(e) < \omega(e')$$

entonces al reemplazar e' por e se obtiene un spanning tree (ST) de costo estrictamente menor que el supuesto MST original $\mathcal{T}$, lo cual es imposible.

Por lo tanto, siempre existe un MST compatible con las decisiones tomadas por Kruskal. El invariante se preserva en cada iteración.

Cuando el algoritmo termina, el conjunto T tiene exactamente $n - 1$ aristas y forma un spanning tree. Además, por el invariante, dicho árbol es compatible con algún MST. En consecuencia, el árbol construido por Kruskal es efectivamente un MST.

Conclusiones

Las demostraciones por inducción aparecen constantemente en diseño y análisis de algoritmos, pero no siempre bajo la misma forma. En recurrencias, la inducción se utiliza para demostrar fórmulas cerradas o cotas asintóticas sobre funciones definidas recursivamente. En algoritmos iterativos, en cambio, la inducción se aplica sobre el estado del algoritmo mediante invariantes de ciclo.

En ambos casos, la idea central es la misma: demostrar que cierta propiedad es verdadera inicialmente y que continúa siendo verdadera después de cada paso del proceso. Sin embargo, entender qué propiedad debe demostrarse y cómo formular correctamente la hipótesis inductiva suele ser la parte más importante y más difícil de la demostración.